

High Level Design

Candida auris Dashboard Österreich

Version 1.04

Studiengang: Computer Science and Digital Communications (CSDC)
Lehrveranstaltung: Wahlfach Projekt 1
Institution: Hochschule Campus Wien
Autorin: Cristina Postoronca, Oyu-Erdene Khurelbaatar, Linlin Kou
Datum: März | April | Juni 2026
Version: 1.04

Dokument-Historie

Version	Status	Datum	Verantwortlicher	Änderungsgrund
1.00	Offen	10.03.2026	Linlin Kou	Erstellung der 1. Version
1.01	Offen	30.03.2026	Linlin Kou	Einarbeitung des Feedbacks von Stakeholder
1.02	Offen	02.04.2026	Linlin Kou	Einarbeitung des Feedbacks von Prof
1.03	Final	10.04.2026	Linlin Kou	Einarbeitung des Feedbacks von Prof & Stakeholder
1.04	Final	20.06.2026	Linlin Kou	Finale Version

Inhaltsverzeichnis

1.	EINLEITUNG	1
1.1.	ZWECK	1
1.2.	SYSTEMÜBERBLICK	1
1.3.	BEZUG ZUM SRS	1
2.	SYSTEMARCHITEKTUR	2
2.1.	ARCHITEKTURÜBERBLICK	2
2.2.	ARCHITEKTURDIAGRAMM	2
2.3.	HAUPTKOMPONENTEN	4
3.	DATENMODELL	4
3.1.	ZENTRALE DATENOBJEKTE	4
3.2.	DATENBANKSTRUKTUR (ER-DIAGRAMM)	5
4.	DATENFLUSS UND SCHNITTSTELLEN	5
4.1.	DATENFLUSS IM SYSTEM	5
4.2.	AKTIVITÄTSDIAGRAMM - MELDEPROZESS	6
4.3.	API / KOMMUNIKATION FRONTEND-BACKEND	7
5.	BETRIEBSUMGEBUNG	7
5.1.	SYSTEMUMGEBUNG	7
5.2.	DEPLOYMENT (HOSTING / SERVER)	7
5.3.	PERFORMANCE	8
6.	SICHERHEITS- UND ZUGRIFFSMODELL	8
6.1.	BENUTZERROLLEN	8
6.2.	ZUGRIFF AUF FUNKTIONEN	8
6.3.	SICHERHEIT / DATENSCHUTZ	9
6.4.	SCHUTZ VOR AUTOMATISierter NUTZUNG	9
7.	ANHANG	10
7.1.	WEITERE TECHNISCHE DETAILS	10

1. Einleitung

1.1. Zweck

Dieses Dokument beschreibt das High Level Design (HLD) des Systems „Candida auris Dashboard Österreich“. Es definiert die grundlegende Systemarchitektur sowie die wichtigsten Komponenten und deren Zusammenspiel.

Das HLD basiert auf den im Software Requirements Specification (SRS) definierten Anforderungen und zeigt, wie diese auf architektonischer Ebene umgesetzt werden.

Das Dokument dient als Orientierung für die weitere Entwicklung und vermittelt einen Überblick über Aufbau, Komponenten und Datenflüsse des Systems.

1.2. Systemüberblick

Das „Candida auris Dashboard Österreich“ ist eine webbasierte Anwendung zur Erfassung, Verwaltung und Darstellung epidemiologischer Daten.

Das System stellt ein öffentlich zugängliches Dashboard bereit und ermöglicht die strukturierte Übermittlung neuer Meldungen über ein Formular. Die übermittelten Daten werden per E-Mail weitergeleitet und nach manueller Prüfung in das System übernommen. Über einen geschützten Verwaltungsbereich können berechnigte Personen die gespeicherten Daten verwalten und aktualisieren.

Die detaillierten funktionalen Anforderungen und Anwendungsfälle sind im SRS-Dokument beschrieben.

1.3. Bezug zum SRS

Dieses High Level Design basiert auf den im Software Requirements Specification (SRS) definierten funktionalen und nicht-funktionalen Anforderungen.

Es beschreibt die grundlegende Systemarchitektur sowie die Komponenten, Datenstrukturen und Kommunikationswege, die zur Umsetzung dieser Anforderungen erforderlich sind.

2. Systemarchitektur

2.1. Architekturüberblick

Die Anwendung „Candida auris Dashboard Österreich“ ist als webbasierte Anwendung mit einer mehrschichtigen Architektur konzipiert.

Ziel dieser Architektur ist eine klare Trennung zwischen Benutzeroberfläche, Anwendungslogik und Datenhaltung.

Das System besteht aus drei zentralen Komponenten: Frontend, Backend und Datenbank. Das Frontend wird als React-Anwendung umgesetzt und stellt die Benutzeroberfläche für Dashboard, Meldeformular und Verwaltungsbereich bereit. Das Backend basiert auf FastAPI und stellt die Anwendungslogik sowie die Schnittstellen zur Datenverwaltung bereit. Die persistente Speicherung der Daten erfolgt in einer PostgreSQL-Datenbank.

Die Komponenten kommunizieren über REST-basierte Schnittstellen und ermöglichen die Erfassung, Verarbeitung, Verwaltung und Darstellung epidemiologischer Daten.

2.2. Architekturdiagramm

Die folgenden Abbildungen zeigen die grundlegende Systemarchitektur sowie den Ablauf der Verarbeitung neuer Meldungen.

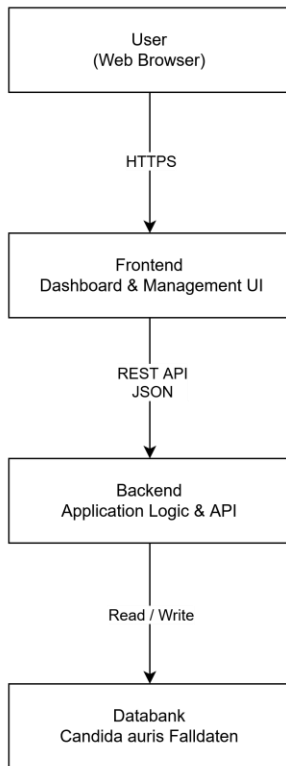
Abbildung A zeigt die technische Architektur des Systems.

Nutzer greifen über einen Webbrowser auf das React-Frontend zu, das über REST-basierte Schnittstellen mit dem FastAPI-Backend kommuniziert. Das Backend stellt die Daten für Dashboard und Verwaltungsbereich bereit und greift auf die PostgreSQL-Datenbank zu.

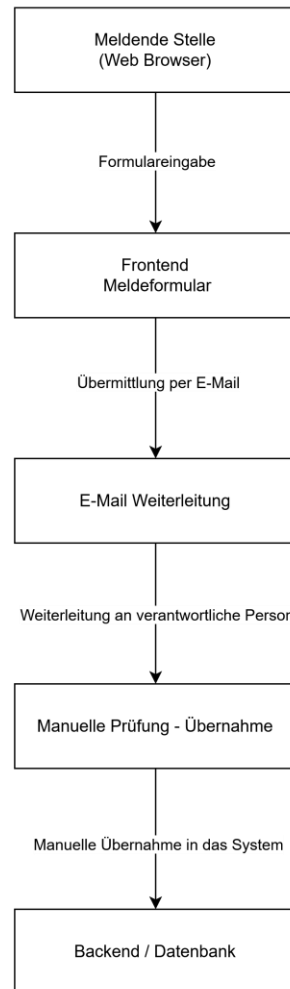
Die REST-basierte Schnittstelle wird für den Abruf, die Verwaltung und die Aktualisierung bereits im System gespeicherter Daten verwendet. Die Übermittlung neuer Meldungen erfolgt bewusst nicht über diese Schnittstelle zur direkten Speicherung, sondern über das Formular mit anschließender Weiterleitung per E-Mail.

Abbildung B zeigt den Ablauf der Übermittlung neuer Meldungen.

Die Eingaben werden über das Formular erfasst, per E-Mail an die verantwortliche Person weitergeleitet und nach manueller Prüfung in das System übernommen.



A - Systemarchitektur für Dashboard und Datenverwaltung



B - Meldeprozess und manuelle Datenübernahme

2.3. *Hauptkomponenten*

Das System besteht aus drei zentralen Komponenten: Frontend, Backend und Datenbank. Diese Komponenten übernehmen unterschiedliche Aufgaben innerhalb der Systemarchitektur und arbeiten über definierte Schnittstellen zusammen.

Frontend

Das Frontend wird als React-Anwendung umgesetzt und stellt die Benutzeroberfläche der Anwendung bereit. Es wird über einen Webbrowser aufgerufen und ermöglicht den Zugriff auf das Dashboard, den Verwaltungsbereich sowie die Eingabe neuer Meldungen. Darüber hinaus ist es für die Darstellung epidemiologischer Daten und Visualisierungen verantwortlich.

Backend

Das Backend basiert auf FastAPI und stellt die zentrale Anwendungslogik bereit. Es verarbeitet die Anfragen des Frontends über die definierten REST-Schnittstellen und übernimmt die Verwaltung, Verarbeitung und Bereitstellung der gespeicherten Daten.

Die Verarbeitung neuer Meldungen erfolgt nicht direkt über diese Schnittstellen. Nach manueller Prüfung durch die verantwortliche Person werden die Daten über das Backend in das System übernommen.

Datenbank

Die Daten werden in einer PostgreSQL-Datenbank gespeichert. Die Datenbank dient zur strukturierten Speicherung der nach manueller Prüfung übernommenen Informationen. Sie stellt sicher, dass die Daten dauerhaft verfügbar sind und für Auswertungen, Visualisierungen und die Datenverwaltung genutzt werden können.

3. Datenmodell

3.1. *Zentrale Datenobjekte*

Für die Umsetzung des Systems werden zentrale Datenobjekte definiert, die die Grundlage für die Speicherung und Verarbeitung der Informationen bilden.

Das zentrale Datenobjekt ist die Meldung. Sie repräsentiert eine nach manueller Prüfung in das System übernommene Fallmeldung und enthält die relevanten epidemiologischen Informationen.

Zu den gespeicherten Informationen zählen insbesondere Angaben zur Einrichtung, Region, Art der Infektion, Lokalisation, Reiseanamnese, Auslandshospitalisierung, Aufnahmegrund bzw. Grunderkrankung, Immunstatus, Therapieinformationen sowie weitere epidemiologisch relevante Zusatzinformationen.

Zusätzlich können genetische Informationen zum Isolat erfasst werden, insbesondere die Zuordnung zu einer Clade (1–6) sowie vorhandene Resistenzdaten.

Diese Datenobjekte ermöglichen eine konsistente Speicherung der Daten und bilden die

Grundlage für Auswertungen, geografische Visualisierungen und weitere Darstellungen im Dashboard.

3.2. Datenbankstruktur (ER-Diagramm)

Die folgende Abbildung C zeigt die vereinfachte Struktur der Datenbank sowie die Beziehungen zwischen den zentralen Datenobjekten.

Das zentrale Element ist die Entität „Meldung“, die mit den Entitäten Einrichtung, Region, Infektionstyp und Lokalisation in Beziehung steht. Darüber hinaus enthält eine Meldung weitere epidemiologisch relevante Informationen wie Reiseanamnese, Auslandshospitalisierung, Aufnahmegrund bzw. Grunderkrankung, Immunstatus, Therapieinformationen, Resistenzdaten sowie gegebenenfalls die Zuordnung zu einer Clade.

Die dargestellte Struktur dient der Veranschaulichung der wichtigsten fachlichen Zusammenhänge und wurde für das High Level Design bewusst vereinfacht dargestellt.

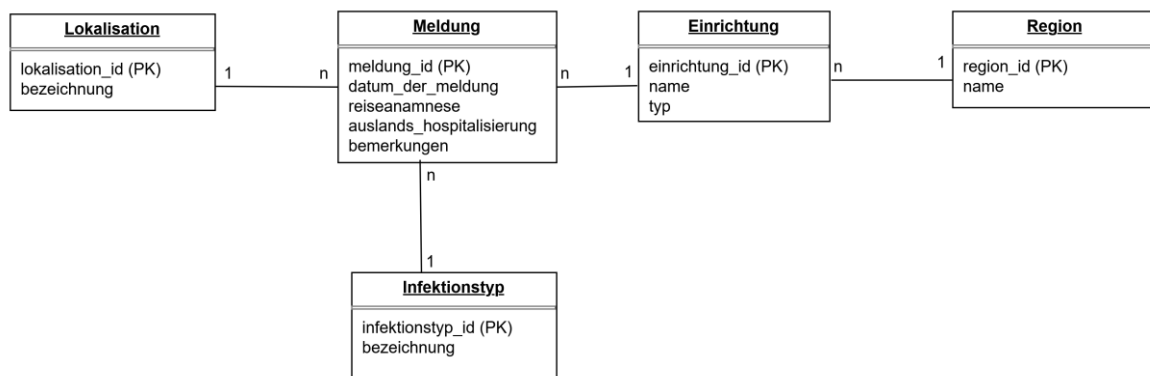


Abbildung C: ER-Diagramm der Datenbankstruktur

Eine Meldung ist jeweils genau einer Einrichtung, einem Infektionstyp und einer Lokalisation zugeordnet. Einrichtungen sind jeweils einer Region zugeordnet.

4. Datenfluss und Schnittstellen

4.1. Datenfluss im System

Der Datenfluss im System erfolgt zwischen Frontend, Backend und Datenbank und umfasst die Anzeige, Verarbeitung und Verwaltung von Daten.

Beim Aufruf des Dashboards fordert das React-Frontend die benötigten Daten über REST-basierte Schnittstellen beim FastAPI-Backend an. Das Backend liest die relevanten Daten aus der PostgreSQL-Datenbank und stellt sie dem Frontend zur Visualisierung zur Verfügung.

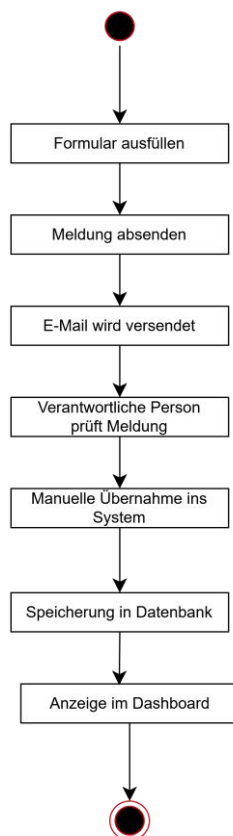
Neue Meldungen werden über ein Formular im Frontend erfasst. Das Formular prüft die Eingaben auf Vollständigkeit und Plausibilität. Anschließend werden die übermittelten Daten an ein definiertes Mailpostfach weitergeleitet. Dort erfolgt die fachliche Prüfung durch die verantwortliche Person. Eine direkte Speicherung im System erfolgt nicht.

Nach fachlicher Prüfung werden die relevanten Daten manuell in das System übernommen. Die Übernahme erfolgt durch eine manuelle Eingabe der Daten in das System. Dabei werden die Daten über das Backend verarbeitet und in der Datenbank gespeichert. Nicht alle im Rahmen einer Meldung erfassten Informationen werden im öffentlich zugänglichen Dashboard dargestellt. Die Anzeige im Dashboard beschränkt sich auf jene Daten, die den Anforderungen an Datenschutz, Anonymisierung und aggregierte Darstellung entsprechen.

Änderungen bestehender Daten erfolgen über den geschützten Verwaltungsbereich der Anwendung und werden über die definierten Schnittstellen an das Backend übermittelt, das die Daten in der Datenbank aktualisiert.

4.2. Aktivitätsdiagramm - Meldeprozess

Das folgende Aktivitätsdiagramm zeigt den Ablauf der Übermittlung und Verarbeitung neuer Meldungen im System, die außerhalb der direkten Kommunikation über die System-Schnittstellen erfolgen.



Aktivitätsdiagramm des Meldeprozesses

4.3. *API / Kommunikation Frontend–Backend*

Die Kommunikation zwischen Frontend und Backend erfolgt über webbasierte REST-Schnittstellen auf Basis von HTTP bzw. HTTPS. Der Datenaustausch erfolgt im JSON-Format.

Die Schnittstellen dienen dem Abruf, der Anzeige und der Aktualisierung bereits im System gespeicherter Daten, insbesondere für die Darstellung im Dashboard und die interne Datenverwaltung.

Die Übermittlung neuer Fallmeldungen erfolgt nicht über diese Schnittstellen zur direkten Speicherung im System. Stattdessen werden neue Meldungen über das Frontend-Formular erfasst und per E-Mail an die zuständige Stelle weitergeleitet. Nach manueller Prüfung werden die Daten durch die verantwortliche Person in das System übernommen.

Das Frontend basiert auf React und kommuniziert über die definierten REST-Schnittstellen mit dem FastAPI-Backend. Die Verarbeitung und Speicherung der Daten erfolgt über das Backend in einer PostgreSQL-Datenbank.

5. Betriebsumgebung

5.1. *Systemumgebung*

Die Anwendung ist als webbasierte Lösung konzipiert und wird über einen Webbrowser aufgerufen.

Das System besteht aus einem React-basierten Frontend, einem FastAPI-Backend sowie einer PostgreSQL-Datenbank zur Speicherung der erfassten Daten.

Das Frontend stellt die Benutzeroberfläche für Dashboard, Meldeformular und Verwaltungsbereich bereit. Das Backend übernimmt die Anwendungslogik, Authentifizierung, Datenverarbeitung und Bereitstellung der Schnittstellen. Die Datenbank dient der persistenten Speicherung der verwalteten Meldungen und zugehörigen Informationen.

5.2. *Deployment (Hosting / Server)*

Das System wird in einer serverbasierten Umgebung betrieben, die den Zugriff auf Frontend, Backend und Datenbank ermöglicht.

Das Frontend wird über eine Weboberfläche bereitgestellt und ist über einen Webbrowser erreichbar. Der Zugriff auf die Anwendung erfolgt über eine öffentlich erreichbare Webadresse (Domain).

Für den produktiven Betrieb ist die Bereitstellung in einer AWS-basierten Cloud-Infrastruktur vorgesehen. Frontend, Backend und Datenbank werden dabei als zusammenhängendes System betrieben.

Während der Entwicklungs- und Testphase kann die Anwendung in einer separaten Hosting-Umgebung bereitgestellt werden. Die Architektur ist so ausgelegt, dass eine spätere Bereitstellung auf AWS unterstützt wird.

5.3. *Performance*

Die Systemarchitektur ist so ausgelegt, dass die im SRS definierten Performance-Anforderungen unterstützt werden.

Das Dashboard soll unter normalen Nutzungsbedingungen innerhalb von maximal **3 Sekunden** vollständig geladen werden.

Die Übermittlung einer Meldung über das Eingabeformular sowie die Weiterleitung per E-Mail sollen innerhalb von maximal **2 Sekunden** erfolgen.

6. Sicherheits- und Zugriffsmodell

6.1. *Benutzerrollen*

Das System unterscheidet zwischen öffentlichen Nutzer:innen und berechtigten Benutzer:innen mit Zugriff auf den Verwaltungsbereich.

Das Dashboard ist öffentlich zugänglich und kann ohne Anmeldung über einen Webbrowser aufgerufen werden.

Für die Pflege und Aktualisierung der Daten ist eine Authentifizierung erforderlich. Der Zugriff auf den Verwaltungsbereich ist ausschließlich berechtigten Personen vorbehalten.

Eine komplexe Rollen- und Rechteverwaltung mit mehreren Hierarchieebenen ist nicht vorgesehen.

6.2. *Zugriff auf Funktionen*

Der Zugriff auf Funktionen zur Pflege und Aktualisierung der Daten ist nicht öffentlich zugänglich und erfolgt ausschließlich durch berechnigte Personen.

Öffentliche Nutzer:innen haben ausschließlich Zugriff auf die im Dashboard dargestellten aggregierten Informationen.

Die Datenpflege erfolgt über einen geschützten internen Bereich der Anwendung. Eine öffentliche Zugänglichkeit dieser Funktionen ist nicht vorgesehen.

6.3. *Sicherheit / Datenschutz*

Im öffentlich zugänglichen Dashboard dürfen ausschließlich aggregierte und anonymisierte Informationen dargestellt werden. Personenbezogene oder sensible Einzeldaten sind nicht sichtbar. Dies umfasst insbesondere, dass keine konkreten Krankenhäuser oder Einrichtungen angezeigt werden. Geografische Informationen werden ausschließlich in aggregierter Form dargestellt, beispielsweise auf Ebene von Bundesländern oder Ländern.

Die im System gespeicherten Daten müssen konsistent und nachvollziehbar verarbeitet werden. Mindestens der Zeitpunkt der letzten Aktualisierung eines Datensatzes wird gespeichert.

Die Sicherheitsmaßnahmen werden durch Authentifizierungs- und Zugriffskontrollmechanismen sowie geeignete technische Schutzmaßnahmen umgesetzt.

6.4. *Schutz vor automatisierter Nutzung*

Das öffentlich zugängliche Meldeformular stellt eine potenzielle Angriffsfläche für automatisierte oder missbräuchliche Nutzung dar.

Das System muss geeignete technische Maßnahmen vorsehen, um eine automatisierte oder massenhafte Übermittlung von Meldungen zu verhindern. Dazu können beispielsweise Captcha-Mechanismen oder vergleichbare Verfahren eingesetzt werden.

Diese Schutzmaßnahmen dienen der Sicherstellung der Datenqualität sowie der Verfügbarkeit des Systems und verhindern eine Überlastung durch automatisierte Anfragen.

Die Schutzmaßnahmen werden im Frontend des Meldeformulars umgesetzt.

7. Anhang

7.1. *Weitere technische Details*

Dieses Dokument beschreibt die Architektur und die zentralen technischen Entscheidungen des Systems auf hoher Ebene. Detaillierte technische Spezifikationen wie konkrete API-Endpunkte, Datenmodelle, Beispielanfragen oder Implementierungsdetails sind nicht Bestandteil dieses High Level Designs.

Diese Informationen werden in den jeweiligen Implementierungsartefakten sowie im Quellcode dokumentiert.

Der Abschnitt kann bei Bedarf um zusätzliche technische Informationen ergänzt werden, beispielsweise zu Schnittstellen, Datenstrukturen oder Deployment-Konfigurationen.